

CS63 Spring 2026

Constructing ResNet-based Fish Classifiers with Transfer Learning and Early Fusion

Nicholas Fettig*

Marlea Martens†

May 6th, 2026

1 Introduction

According to the creators of FishBase [4], the world’s largest publicly available fish database, fish represent the most diverse vertebrate group on Earth. Fish, with over 34,800 described species as of 2022, exceed the combined total species count of all mammals, amphibians, reptiles, and birds. This profound scale of fish diversity makes their classification a monumental yet vital task for purposes of species preservation, seafood labeling, and study of fish ecology. A lack of properly verifiable fish identification methods directly also impacts consumers. Particularly unsettlingly, the Food and Agriculture Organization (FAO) estimates that up to 20% of the \$195 billion global seafood trade may be subject to food fraud [3] – deliberate mislabeling or species substitution intended to deceive consumers – a rate higher than any other food sector. Implementing technology that properly labels fish benefits producers and consumers alike.

Given the scope and importance of fish classification, there exists plenty of large-scale open-source datasets. The aforementioned FishBase contains brief descriptions and a few pictures for all 34,800 known species. Fish4Knowledge focuses on high-resolution videos with labeled bounding boxes for 23 common species [1]. DeepFish contains population and localization information for all fish off the coast of Australia [9]. Given our limited scope of time and compute, our main focus for this project is classification of high quality images with a low total species count. Inspired by the the seafood mislabeling problem, we decided on a 9-species dataset with species caught specifically for human-consumption: A Large-Scale Dataset for Fish Segmentation and Classification, arranged by Ulucan et al. [11].

From preliminary research in Kaggle forums and academic papers, we found that low-species count models are typically able to obtain test-set accuracies of about 90-99%. We drew particular inspiration from Murugaiyan et al. [7] who were able to leverage transfer learning off of Google Inception-v3, a pretrained ImageNet model, to obtain 95.37% accuracy on Fish4Knowledge (in 2021, no less). Given that our chosen dataset is only of 9 species and that ImageNet-based models have advanced significantly in the last five years, we sought to achieve a similar accuracy.

We designed and implemented two models to learn this dataset, both leveraging a ResNet50-based architecture. The first model implements transfer learning from ImageNet while the second

*nfettig1@swarthmore.edu

†mmarten1@swarthmore.edu

model implements early fusion on image masks already provided in the dataset. The transfer learning model was able to achieve 89.89% accuracy while the early fusion model achieved 97.22% accuracy over 20 epochs each. Considering the strengths and weaknesses of each model and the availability of information provided in our chosen dataset, we believe the early fusion model is a better option for the fish classification task.

2 Dataset

The Large-Scale Fish Dataset (LSFD) was originally introduced as an alternative to other widely used fish datasets, which often prioritize the capture of many underwater species as opposed to species specifically for human consumption. Through sampling popular seafood types from a supermarket in Izmir, Turkey, LSFD avoids the blue filter often seen in underwater images and also ensures that the samples are directly applicable to human consumption habits. [11].

FISH	ABBREVIATION	LABEL	COUNT	PERCENT
BLACK SEA SPRAT	BSS	0	1000	11.1%
GILT-HEAD BREAM	GHB	1	1000	11.1%
HOARSE MACKEREL	HM	2	1000	11.1%
RED MULLET	RM	3	1000	11.1%
RED SEA BREAM	RSB	4	1000	11.1%
SEA BASS	SB	5	1000	11.1%
SHRIMP	S	6	1000	11.1%
STRIPED RED MULLET	SRM	7	1000	11.1%
TROUT	T	8	1000	11.1%

Table 1: Large-Scale Fish Dataset species counts, abbreviations, and labels as used in our implementation.

In total, there are 9000 RGB images and 9000 grayscale images in the dataset. To add diversity to the training data, LSFD also includes repeated and rotated images of the same fish. This is useful for machine learning classifiers, as this introduces diversity to help the model generalize to unseen data. To further add randomness, our classifier also utilizes image augmentations like rotation (± 10 degrees), flip (horizontal), brightness (20%), and contrast (20%). Additionally, the images are paired with a corresponding segmentation mask (that is, for every image I , there is also a matching I_{mask}). The only metadata provided is the fish species type, of which was translated to labels in the range (0,8). Table 1 summarizes labeling and counts for each species in LSFD.

3 Methods

3.1 Data Pre-processing

For the base CNN, the images were stacked and formed into tensors after rescaling dimensions and normalizing RGB channels for ResNet. Augmentations including rotation (± 10 degrees), flip (horizontal), brightness (20%), and contrast (20%) were also applied.

The image pre-processing methods for the CNN with segmentation followed a somewhat similar normalization strategy for ResNet. Images were similarly stacked and formed into tensors after

rescaling dimensions and normalizing RGB channels for ImageNet. However, before augmentation, we expanded the dimensions of the mask images (grayscale images are two dimensional, as opposed to RGB images, which are three dimensional), concatenated the images and masks, and lastly applied the same augmentations (excluding brightness and contrast) listed above. This pipeline ensured that any transformations like horizontal flips would apply to both the input image and mask. We excluded brightness and contrast, as this would alter the black and white format of the mask, which could lead to confusion for the model. An example for each species and it's mask can be provided in Figure 1.

Random Examples from Each Fish Class

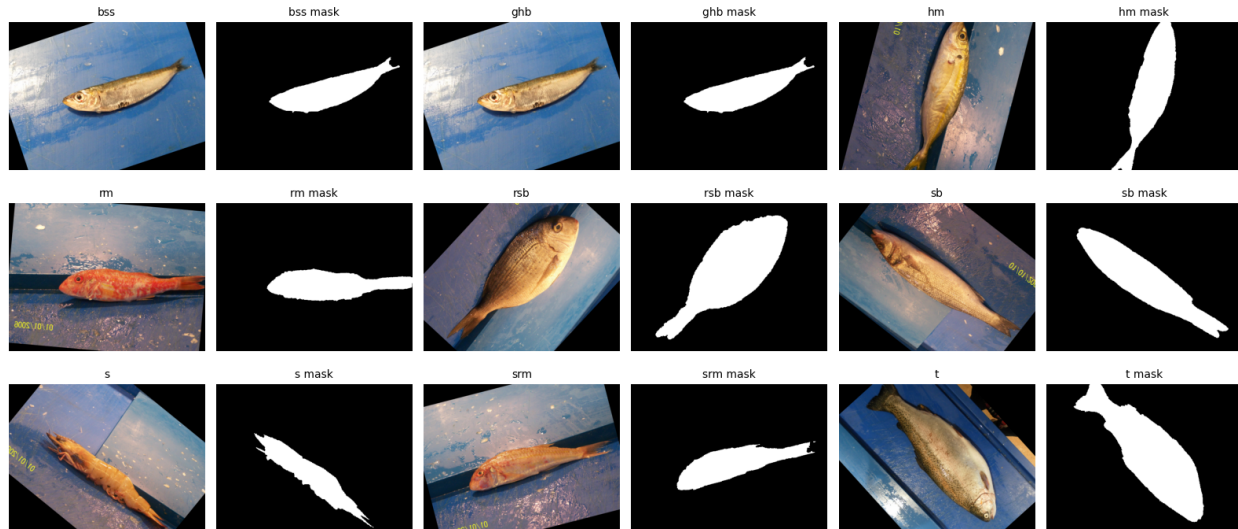


Figure 1: Grid of LSFD example images with their corresponding masks.

3.2 Training and Validation Sets

Data was split into two sets for training and validation. We used scikit-learn’s `TRAIN_TEST_SPLIT`, with 90% of the data dedicated for training, and 10% dedicated for validation/testing. We used the stratification flag on labels to maintain an even split of each fish in training and validation sets. We also used a set seed in order to reproduce the same training/testing sets so we can compare training runs with each other.

We allocated a large proportion of data towards training because ResNet50 has many layers and we are training it from scratch in one of our experiments. The ResNet model we chose contains nearly 25M parameters meaning it’s prone to underfitting on small datasets, so our strategy was to maximize how much data we are feeding the model during training. Of course, this is a tradeoff since we are testing a less amount of data and will therefore have a less accurate representation of how well our model is performing.

3.3 Introduction to Deep Learning & Convolutional Neural Networks (CNN)

Deep learning is a machine learning method which utilizes a network of nodes and weights to transform data to produce a prediction. The difference between the prediction and ground truth is backwards propagated to adjust the network’s existing weights and reduce future error. In backwards propagation, each weight is updated proportionally to the partial derivative of the error function with respect to that particular weight. An activation function (e.g. ReLU or sigmoid) is then used to decide whether a neuron should turn on, introducing non-linearity and allowing the model to learn complex relationships from the data.

In deep learning models, two-dimensional images must be flattened before being used as input. Consequently, all spatial relationships between pixels are lost. Convolutional Neural Networks address this fundamental gap in standard feedforward networks’ ability to process image data. Through a series of specialized layers for processing two-dimensional images, spatial relationships are preserved, resulting in highly accurate image predictors. Specialized layers include convolution, pooling, and flattening layers. Convolution layers learn kernels which can be applied to small regions of the input image, detecting features like straight lines, fur texture, curvature, etc. Pooling reduces dimensionality by summarizing patches of images with single values, and flattening layers convert two-dimensional representations of data into one-dimensional ones.

3.4 ImageNet Residual Network (ResNet50)

ResNet50, created by a team of researchers at Microsoft in 2015, is a convolutional neural network with 50 layers and multiple stages of bottleneck blocks, a unit designed to make the system more computationally efficient [6]. The first stage introduces convolution layers, which utilize the ReLU activation function and help with feature extraction. Images are then passed to a series of convolution and identity blocks, which help with high level feature extraction. Lastly, fully connected layers are used to make predictions based on previously extracted features, utilizing the softmax activation function. One of the most important contributions of ResNet50 was the innovation of residual blocks, which help mitigate errors associated with the vanishing gradient problem [5]. Residual blocks connect the activation of previous layers with future layers while skipping some layers in between, preserving the vectors of partial derivatives.

The network was designed with ImageNet in mind; a large publicly available image dataset for computer vision research [2]. As a result, input images for ResNet50 must be in the dimensions of 224×224 pixels. This foundational relationship with ImageNet is precisely what makes ResNet50 so effective for transfer learning. ImageNet has over 14 million images, which ResNet50 was trained on. Consequently, ResNet50 is able to detect a wide range of features, from low level shapes like corners and edges, to high level patterns like fur and scales. By saving the trained weights and replacing the final dense layer, new models made for specific prediction purposes can capitalize on these pre-learned relationships. This allows for researchers to save both time and resources, as transfer learning mitigates the need to train networks completely from scratch [10].

An image representation of the ResNet50 architecture can be found in Figure 2.

3.5 Early Fusion

Early fusion is a child of multimodal deep learning, first introduced by Ngiam et al. [8] in 2011. Fusion, in the context of ML, refers to combining multiple input modalities to improve a network’s behavior. Early fusion means we are combining inputs *before* entering the network, which seemed

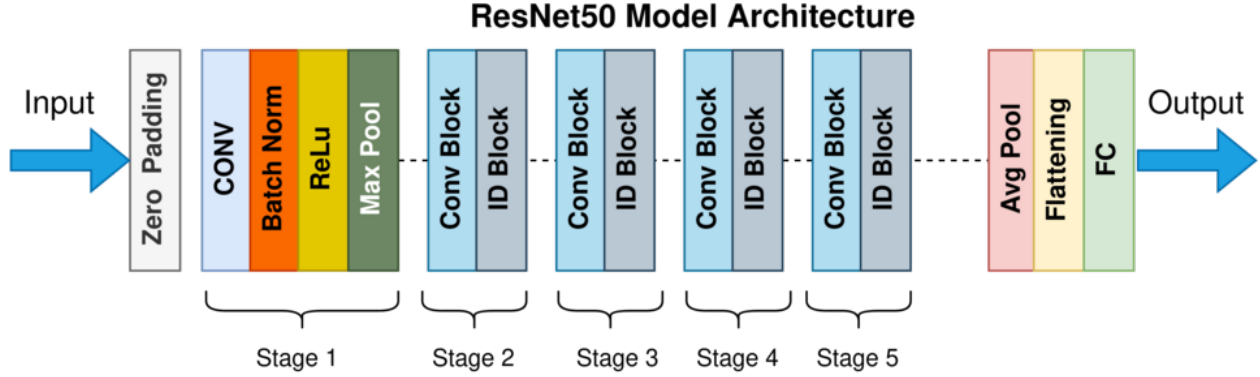


Figure 2: A visual depiction of ResNet50’s architecture. [6].

most appropriate when considering the image data available in LSFD. In the context of our experiments, we can feed a 4-channel input into a neural network: the first three channels sourced from the RGB image and the last channel being the image mask. We are hoping that the additional input layer will provide additional contours for the feature detector to pick up on in training.

3.6 Hyperparameters

Given the scale of our network (50+ layers from ResNet50) and the size of our dataset (9000/18000), we were able to get away with a surprising small number of hyperparameters. Our chosen hyperparameters are described below:

- Batch Size: The number of images the model sees before updating it’s weights.
- Epochs: How many times the model trains over the full dataset.
- Learning rate: magnitude of weight update as used in backpropagation.

We also employ a learning rate scheduler that reduces the learning rate over time. This is included as a callback function in our model after each training step. This sort of follows the exploration-exploitation paradigm we talked about in class. More specifically, our learning rate scheduler callback decays the learning rate by half every 10 epochs.

4 Experiments

4.1 Base Experimental Architecture

The purpose of our experiments is to maximize accuracy using a pre-built architecture, ResNet50. Therefore, we put together two experiments building upon the same default architecture. The base of this architecture is ResNet50. The head of this architecture, that being the layers following ResNet50, is twofold: a dense 256 node layer and a dense 9 node layer. The last layer is a softmax probability distribution over the 9 fish species. We chose to add two layers following ResNet50 so we could "freeze" ResNet50 weights in the first of our experiments. Freezing ResNet50 allows us to

leverage the feature extraction capabilities pretrained weights from ImageNet and focus on training the head of our network instead.

For our loss, we use `SPARSE_CATEGORICAL_CROSSENTROPY` and for our optimizer we use `ADAM`, as provided by Tensorflow. We also implement dropout which randomly disables neurons during each training step. We use a dropout value of 0.5 in both experiments, meaning 50% of neurons are randomly disabled at each step. This prevents our model from becoming reliant on specific neurons, improving generalizability in the network. Both experiments also leverage the same hyperparameters:

- `BATCH_SIZE` = 128
- `EPOCHS` = 20
- `LEARNING_RATE` = 0.001

These hyperparameters were chosen to keep training short (within a couple hours) since we were paying for a limited amount of cloud compute tokens on Google Colab. With more time and compute, we'd likely go for a larger batch size, more epochs, and a less intense learning rate scheduler.

4.2 Experiment 1: ImageNet Transfer Learning

The purpose of this experiment is to leverage pre-trained weights from ResNet50 for ImageNet. It's worth mentioning that these weights take up 94.8 MB of disk space to download. As aforementioned, we freeze these weights to maintain ResNet50's pre-trained feature extraction capabilities. We only care about training the weights of the head layers in this experiment. This also means that dropout is *only* applied to the head layers, which ends up having meaningful ramifications in our analysis of the loss and accuracy results later.

4.3 Experiment 2: Early Fusion with Fish Masks

The purpose of this experiment is to leverage *all* data provided from the LSFD in our network. We employ early fusion with the standard RGB images as well as the mask layer to create a four-channel input layer. In this case, we cannot use transfer learning because ImageNet weights only work on 3-channel (RGB image) input. This means that we must attempt to train ResNet50 with random initial weights and without freezing.

5 Results & Discussion

5.1 Experiment 1: ImageNet Transfer Learning

The first experimental model achieved a best accuracy of 89.89% over 20 epochs. The accuracy and loss history of it's training can be seen in Figure 3. It's immediately noticeable that the validation set appears to perform much better than the training set which seems unusual at first. But this is simply a result of dropout being applied to half of the neurons in the training set evaluation. Dropout is only applied on one layer (the 256 dense layer in the head), so it makes sense why the network would struggle to achieve great accuracy/loss values on the training set, even if it's training correctly. We are certain however that the model trained well since the validation set continues to

improve over the 20 epochs. A validation set accuracy of 89.89% nearly falls within the 90-99% accuracy range that we were expecting.

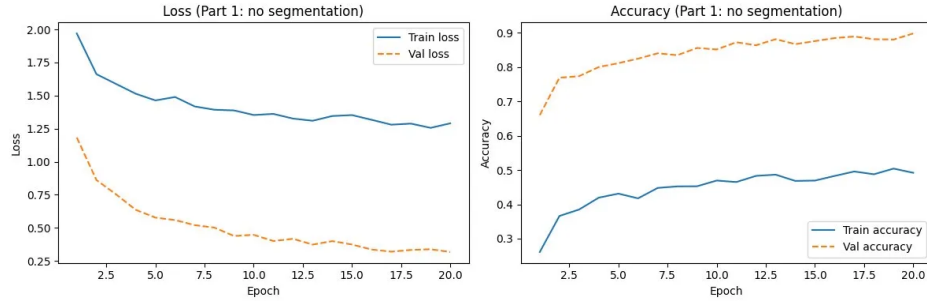


Figure 3: The history of accuracy/loss during training of the first experiment: ImageNet Transfer Learning.

The confusion matrix for this experiment, found in Figure 4, is a good indicator of where the model struggles. The model does quite well all around, except for with the Striped Red Mullet classifier where it only achieves an accuracy of 52%. Visually (and somewhat naively, since we are not fishermen), the Striped Red Mullet appears to be the *least* distinct species in the list. It is closely related to the Red Mullet, another fish species in the dataset, which was challenging for the model to discern. What’s most surprising is that the model most often confused the Red Striped Mullet with Shrimp, which was perhaps the *most* distinct species in the dataset. We were happy to see every other species perform well (over 90% accuracy) which falls in the range of what we find acceptable for our model.

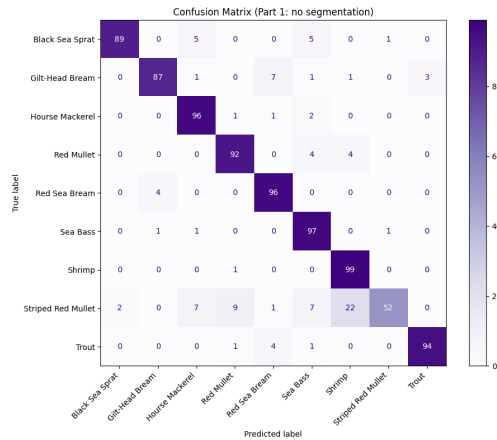


Figure 4: The confusion matrix on the best result of the first experiment: ImageNet Transfer Learning.

5.2 Experiment 2: Early Fusion with Fish Masks

The second experimental model achieved a best accuracy of 97.22% over 20 epochs. The accuracy and loss history of it’s training can be seen in Figure 5. The model converges *very* quickly and

shows no significant improvement after 10 epochs which was a surprise. Recall that we are training each of the ResNet50 weights from scratch, so we assumed it would take longer to train the model both in time and in epochs. The first few epochs took longest, so we assume once the model found weights that got above 90% accuracy, the model spent less time tuning ResNet50 and more time improving the head, but we would need to run more targeted experiments to prove this theory. Also notice that unlike the first experiment, the training accuracy always outperforms the validation accuracy, even with dropout. Given that dropout is applied over 51 layers (as opposed to one layer before), we believe that this gives more time for information to "correct itself" in the network before entering the softmax layer. Our model falls well within the expected 90-99% accuracy range and avoids overfitting to the training set.

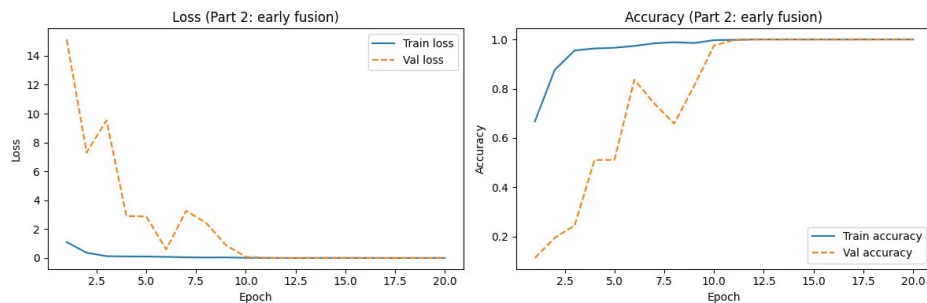


Figure 5: The history of accuracy/loss during training of the second experiment: Early Fusion with Fish Masks.

The confusion matrix for the second experiment, found in Figure 6 shows very little struggle in the model. Like the first experiment, the model struggled most with the Striped Red Mullet, but still maintains the 90%+ accuracy we were hoping for. Somewhat surprisingly, it confuses the species most with Sea Bass which was not so in the first model. Nevertheless, improvement on the model should be directed towards the Striped Red Mullet.

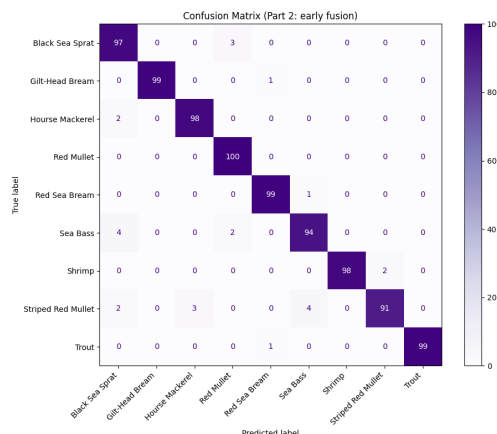


Figure 6: The confusion matrix on the best result of the second experiment: Early Fusion with Fish Masks.

5.3 Discussion of Experiments

Both experiments demonstrated notable strengths and weaknesses. For the model using ImageNet transfer learning, training took 19:42 to complete, while the model utilizing early fusion took 23:12. Although the times were somewhat close (having a difference of only 3 minutes and 36 seconds), it can be argued that with a larger dataset, this training time can become non-negligible.

Building on the first experiment, it is important to note resource overhead. Importing the ResNet50 pre-trained weights requires a large amount of disk space, needing 94.8 MB to download. In a research environment, this space is trivial; however, in the case that this model needs to be deployed for users with limited compute available (e.g. fisherman on boats without access to research-grade technology), this could prove challenging.

Early fusion also introduces its own factors for consideration. Data pre-processing introduced a lot of overhead before model creation even began. Luckily, the LSFD provided us with corresponding black and white masks for each fish. However, in the case where this was not already provided, we would have spent many hours just masking out fish profiles. Additionally, the introduction of grayscale silhouettes as a fourth dimension meant that we could no longer use color-changing alterations in our image augmentation suite. This decreases the amount of diversity and randomness the model can learn from, which may impact its ability to generalize to unseen positions or lighting.

It is also important to consider the implications of using ResNet50 as a base model. Because ResNet50 was built with ImageNet in mind, input images must be resized to 224×224 pixels. This downsampling may result in the loss of spatial information, impacting the model’s ability to detect smaller features and patterns.

For both experiments, dropout had a positive impact on the models’ ability to generalize, and prevented the networks from becoming overly reliant on specific neurons. Both models also displayed great performance on most fish. Excluding the striped red mullet, the first network performed with an average accuracy of 93.5%, and the second network with an average accuracy of 98%.

6 Conclusions

Overall, we found that a standard ResNet50 based CNN with transfer learning achieved 89.89% accuracy. In contrast, the CNN with early fusion performed at a much higher accuracy of 97.22%. Notably, both models exhibited similarity in the categories they struggled with classifying. The Striped Red Mullet proved to be the most confusing fish species, demonstrating the lowest accuracy rates across all possible labels. Regardless, we feel that the CNN with early fusion showed better performance overall and we were satisfied with the results from both models.

With more time, we’d focus heavier on tuning hyperparameters to yield higher accuracy rates. Specifically, it would be interesting to experiment with a larger number of training epochs and a slower learning rate scheduler. It would also be interesting to try our architecture on a different fish datasets with more species and diversity in photo quality.

7 Acknowledgments

We thank Prof. Lisa Meeden for her guidance in the ideation and execution of our experiments, and also for a great semester in CS63! We include all relevant literature cited in the section below.

We also leveraged Claude Sonnet 4.6 for much of the model implementation. We are very clear in code comments what parts of the notebook were written by AI and what parts were written by hand. This report was written entirely by us.

8 References

- [1] Bastiaan J Boom, Jiyin He, Simone Palazzo, Phoenix X Huang, Hsiu-Mei Chou, Fang-Pang Lin, Concetto Spampinato, and Robert B Fisher. A research tool for long-term and continuous analysis of fish assemblage in coral-reefs using underwater camera footage. *Ecological Informatics*, 2014.
- [2] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. pages 248–255, 2009.
- [3] Food and Agriculture Organization of the United Nations. Food fraud in the fisheries and aquaculture sector. Fisheries and Aquaculture Technical Paper 742, FAO, Rome, Italy, 2026.
- [4] Rainer Froese and Daniel Pauly. FishBase. World Wide Web electronic publication, 2026. Accessed: February 2026.
- [5] Mostafa Ibrahim. The basics of resnet50. *Weights and Biases by CoreWeave*, 2024.
- [6] Suvaditya Mukherjee. The annotated resnet-50. *Towards Data Science*, 2022.
- [7] Swamynathan Murugaiyan et al. Fish species recognition using transfer learning techniques. *International Journal of Advances in Intelligent Informatics*, 7(2):188–197, 2021.
- [8] Jiquan Ngiam, Aditya Khosla, Mkhail Kim, Juhan Nam, Honglak Lee, and Andrew Y Ng. Multimodal deep learning. In *Proceedings of the 28th International Conference on Machine Learning (ICML)*, 2011.
- [9] Alzayat Saleh, Issam H Laradji, Dmitry A Konovalov, Michael Bradley, David Vazquez, and Marcus Sheaves. A realistic fish-habitat dataset to evaluate algorithms for underwater visual analysis. *Scientific Reports*, 10(1):14671, 2020.
- [10] Ritesh Sinha. Transfer learning using resnet. *Kaggle*, 2021.
- [11] Oguzhan Ulucan, Diclehan Karakaya, and Mehmet Turkan. A large-scale dataset for fish segmentation and classification. In *2020 Innovations in Intelligent Systems and Applications Conference (ASYU)*, pages 1–5. IEEE, 2020.